Link-Cut деревья

Постановка задачи

Пусть у нас есть несколько деревьев с корнем $t_1, \ldots, t_k$. Хочется поддерживать следующие операции

- `link(u, v)` — пусть $u \in t_i$, $v \in t_j$, причем v является корнем дерева; приклеиваем дерево t_j к дереву t_i, добавляя ребро от u к v,
- `cut(u)` — пусть $u \in t_i$ и не является корнем; отрезаем поддерево u от дерева t_i, убирая ребро между u и $\mathrm{parent}(u)$.

Хотим делать все эти операции за $O^*(\ln(n))$ [т.е. за амортизированное логарифмическое время].

[На самом деле хочется уметь делать чуть больше, но пока сконцентрируемся на этих].

Как устроена структура данных?

Пусть $t = t_i$. Каждое ребро t назовем либо *сплошным*, либо *пунктирным*, причем из каждой вершины вниз выходит не более одного сплошного ребра [изначальное разделение на сплошные и пунктирные ребра может быть произвольным; структура данных потом решит как ей удобно закрашивать ребра]. В итоге дерево t распадается в дизъюнктное объединение сплошных путей [возможно, состоящих только из одной вершины]. Каждый сплошной путь будем хранить в виде splay-дерева, порядок на вершинах пути согласован с расстоянием до корня [на самом деле никакие ключи для splay-дерева мы хранить не будем, но мы будем *неявно* поддерживать такой порядок в splay-деревьях будто бы ключами являются расстояния $k(u) = \mathrm{dist}(u, r)$, где r — корень дерева t]. Пока получается как на Рис. 1.

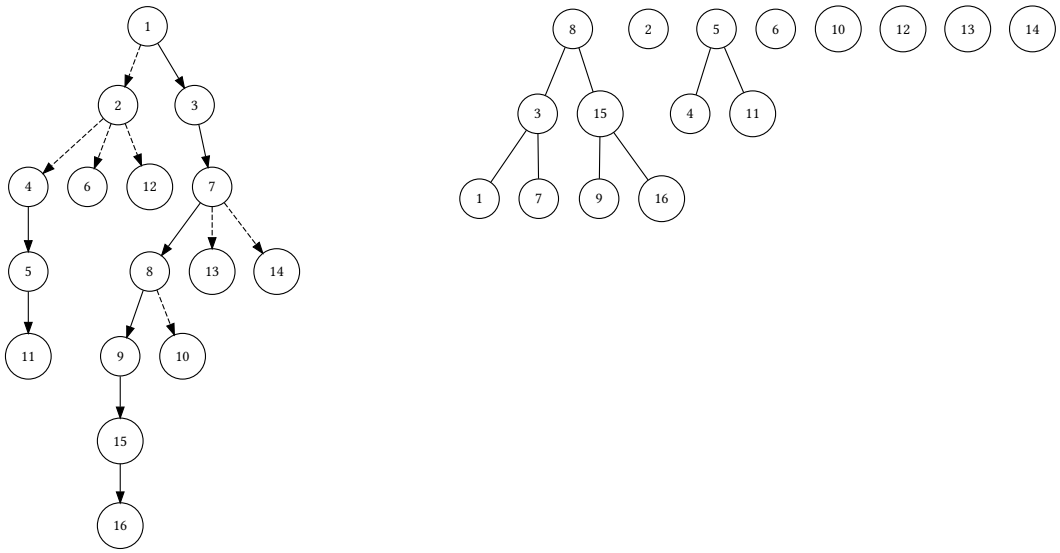


Рис. 1. Пример хранения путей в виде splay-деревьев

Пока splay-деревья путей между собой не были никак связаны — исправим это. Пусть в дереве t путь p подвешивается за вершину u, тогда добавляем указатель из корня splay-дерева пути p на вершину u [на рисунке обозначается пунктирным неориентированным ребром]. Этот указатель будем называть $\mathrm{pathparent}(v) = u$, где v — корень splay-дерева для пути p. Получается как на Рис. 2.

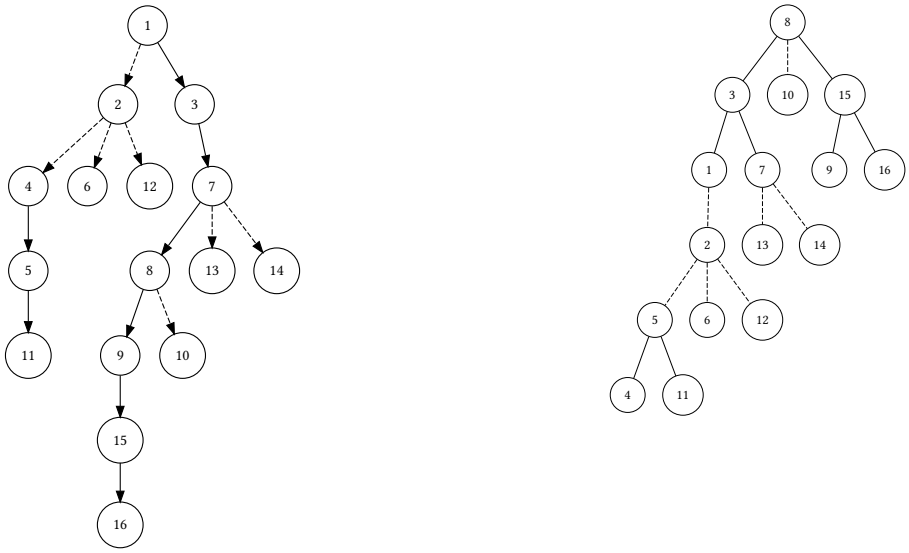


Рис. 2. Пример хранения путей в виде splay-деревьев с иерархией

Получившаяся каша из splay-деревьев называется *виртуальным деревом для дерева* t. У каждой вершины u виртуального дерева есть левый и правый дети, а также средние дети — вершины, из которых есть указатель на u [но сама вершина про средних детей мало что знает].

Как реализуются операции?

Пусть $t = t_i$. Сначала научимся делать вспомогательную операцию `expose(u)`, которая переназначает сплошные/пунктирные ребра так, что u и r являются концами одного пути, где r — корень дерева t. См. Рис. 3.

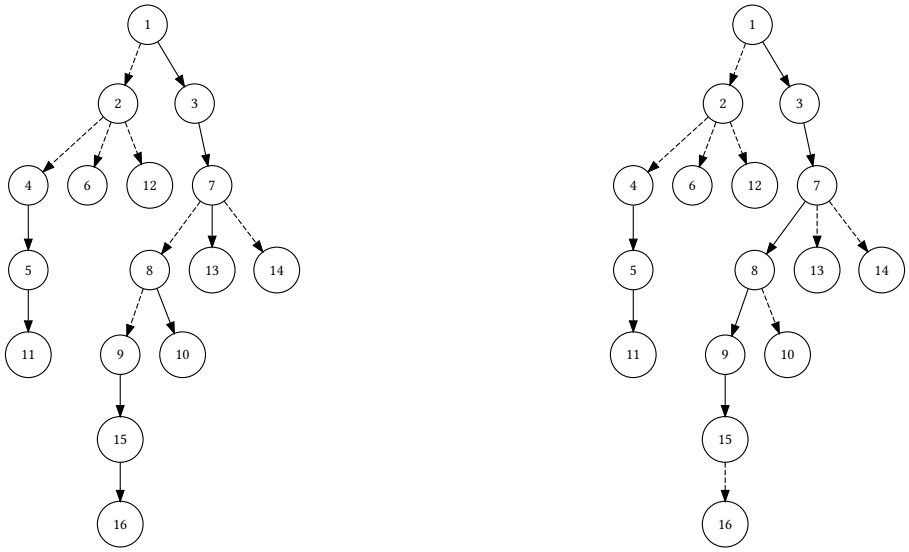


Рис. 3. Пример исполнения expose(15)

Реализуем в три шага:

1. **Splaying**. Делаем $\mathrm{splay}(u)$, $\mathrm{splay}(\mathrm{pathparent}(u))$, $\mathrm{splay}(\mathrm{pathparent}(\mathrm{pathparent}(u)))$, ... пока не упремся в корень дерева t.
2. **Splicing**. Отрезаем правого потомка $\mathrm{pathparent}(u)$ и вместо него подвешиваем splay-дерево с корнем в u, отрезаем правого потомка $\mathrm{pathparent}(\mathrm{pathparent}(u))$ и вместо него подвешиваем splay-дерево с корнем в $\mathrm{pathparent}(u)$, ... пока не дойдем до корня.

3. **Splay опять**. Делаем splay(u) [к этому моменту u и r будут в одном splay-дереве], [отмечу также, что этот шаг важен для амортизационного анализа, без него бы не получилось логарифмическое время].

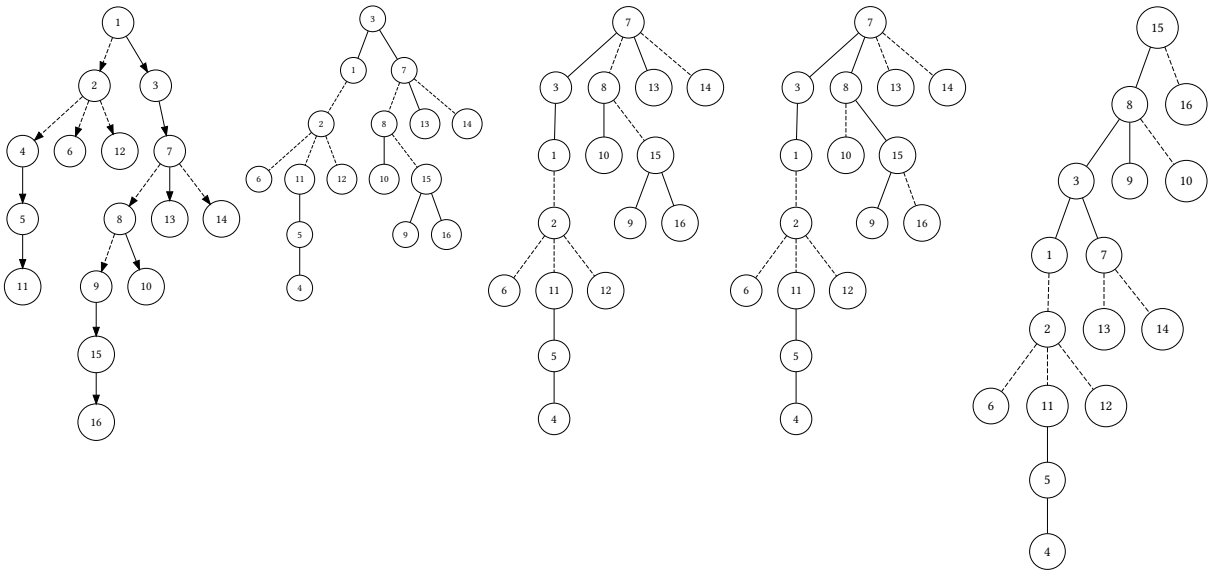


Рис. 4. Пример исполнения expose(15) по шагам

Далее будет показано, что expose работает за $O^*(\ln(n))$. Поэтому достаточно выразить все операции в терминах expose.

- `link(u, v)` — пусть $u \in t_i$, $v \in t_j$; expose(u), правого поддерева у u не будет, поэтому достаточно присвоить pathparent$(v) = u$,
- `cut(u)` — expose(u), правого поддерева у u нет, поэтому достаточно присвоить pathaparent$(u) = \text{null}$.

Почему работает быстро?

Определение 1 Назовем ребро uv легким, если $s(u) \leq \frac{1}{2}s(v)$ [v считаем родителем u], где $s(u)$ — количество вершин в поддереве вершины u в дереве t.

Лемма 2 Для любой вершины $u \in t$ количество легких ребер на пути от u до корня r не превосходит $\log_2(n)$. В противном случае будем называть ребро тяжелым.

Доказательство. Пусть m — количество легких ребер на пути $[u, r]$. Рассмотрим путешествие по пути $[u, r]$; каждый раз когда мы проходим по легкому ребру количество вершин в поддереве увеличивается хотя бы вдовое. В итоге после путешествия до корня во всем дереве должно быть хотя бы 2^m вершин, но $2^m \leq n \Leftrightarrow m \leq \log_2(n)$. □

Лемма 3 (Напоминание из доклада про splay-деревья) Каждый splay(u) в splay-дереве работает за $\leq 1 + 3(\text{rk}'(u) - \text{rk}(u))$, где $\text{rk}(u)$ — ранг вершины u, $\text{rk}'(u)$ — ранг вершины u после выполнения splay [под рангом здесь понимается

$$\text{rk}(u) = \sum_{v \text{ является потомком } u \text{ в splay-дереве}} w(v)$$

где $w(v) \geq 0$ — некоторые веса вершинок в виртуальном дереве; никого отношения к структуре данных они не имеют, просто полезная вещь для амортизационных оценок].

Лемма 4 expose(u) работает за $O^*(\ln(n))$.

Доказательство. Воспользуемся методом потенциалов. В итоге хотим показать, что

$$O(k\ln(n)) \geq \sum_{j=1}^{k} T_j^* = \sum_{j=1}^{k} (T_j + \Delta\Phi_j)$$

где k — количество операций expose.

Рассмотрим функцию потенциала $\Phi(S) = \#$тяжелых пунктирных ребер [т.е. мерим насколько наша декомпозиция на сплошные/пунктирные ребра похожа на HL(heavy-light)-декомпозицию; если сильно похожа — потенциал маленький, но тогда и запросы будут происходить быстро].

Рассмотрим путь от u до r — корня дерева t. Пусть на этом пути было ℓ пунктирных ребер, причем x из них являются тяжелыми, y из них являются легкими.

Тогда $\Delta\Phi_j \leq -x + y$: потенциал точно уменьшился на x [так как x тяжелых пунктирных ребер заменили на сплошные], и увеличился не более чем на y [так как после перекрашивания пунктирного в сплошное, возможно, пришлось перекрасить тяжелое сплошное — так как у каждой вершины может быть не больше одного сплошного исходящего ребра].

Также заметим, что [при правильном выборе весов для ранговой функции: хотим, чтобы $\mathrm{rk}(\mathrm{pathparent}^{i-1}(u)) - \mathrm{rk}'(\mathrm{pathparent}^{i}(u)) \geq 0$]

$$T_j \leq (1 + 3(\mathrm{rk}'(u) - \mathrm{rk}(u))) + (1 + 3(\mathrm{rk}'(\mathrm{pathparent}(u) - \mathrm{rk}(\mathrm{pathparent}(u))))) + \ldots \leq \ell + 3(\mathrm{rk}(r) - \mathrm{rk}(u))$$

Откуда $T_j + \Delta\Phi_j \leq x + y + 3(\mathrm{rk}(r) - \mathrm{rk}(u)) + (-x + y) = 2y + 3(\mathrm{rk}(r) - \mathrm{rk}(u)) \leq 8\log_2(n)$.
Откуда $\sum T_j + \Delta\Phi_j \leq 8k\ln(n) = O(k\ln(n))$. □

Как к этому прикрутить Min & Range-Update?

Пусть каждая вершина $u \in t_i$ имеет некоторый вес, будем обозначать его $\mathrm{cost}(u) \in \mathbb{R}$. Пусть мы еще хотим поддержать операции

- `min(u)` — пусть $u \in t_i$, r — корень дерева; вычисляем $\min_{v \in [r,u]} \mathrm{cost}(v)$ [т.е. минимум на пути от вершины u до корня r],
- `add(u, x)` — пусть $u \in t_i$, r — корень дерева; прибавлять к весу каждой вершины пути $[u, r]$ значение x.

Тогда в каждой вершине виртуального дерева также будет хранить $\Delta\mathrm{cost}(u)$, $\Delta\min(u)$, определяемые формулами на Рис. 5

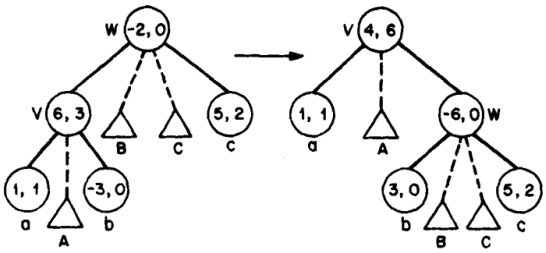
$$\Delta cost(x) = \begin{cases} cost(x) & \text{if } x \text{ is the root of a solid subtree of } V, \\ cost(x) - cost(p(x)) & \text{otherwise;} \end{cases}$$

$$\Delta min(x) = cost(x) - min\ cost(x).$$

Остается только пересчитывать $\Delta\mathrm{cost}$, $\Delta\min$

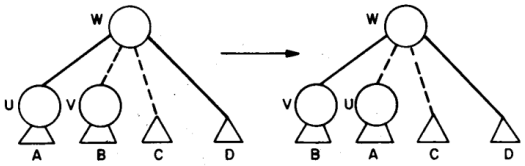
1. при вращениях в splay-деревьях

$$
\begin{aligned}
\Delta cost'(v) &= \Delta cost(v) + \Delta cost(w),\\
\Delta cost'(w) &= -\Delta cost(v),\\
\Delta cost'(b) &= \Delta cost(v) + \Delta cost(b),\\
\Delta min'(w) &= \max\{0,\ \Delta min(b) - \Delta cost'(b),\ \Delta min(c) - \Delta cost(c)\},\\
\Delta min'(v) &= \max\{0,\ \Delta min(a) - \Delta cost(a),\ \Delta min'(w) - \Delta cost'(w)\}.
\end{aligned}
$$



2. при link/cut'ах в виртуальных деревьях

$$
\begin{aligned}
\Delta cost'(v) &= \Delta cost(v) - \Delta cost(w),\\
\Delta cost'(u) &= \Delta cost(u) + \Delta cost(w),\\
\Delta min'(w) &= \max\{0,\ \Delta min(v) - \Delta cost'(v),\ \Delta min(right(w)) - \Delta cost(right(w))\}.
\end{aligned}
$$



Тогда операции реализуются так:

- `min(u)` — expose(u), вернуть cost(u) − Δmin(u)
- `add(u, x)` — expose(u), увеличить cost(u) и Δcost(u) на x

все магически будет пересчитываться при expose(u).